TokTickIT Service Desk — Lab 4 Submission Report

Source: Official CPE 334 Lab 4 handout, `SE+Lab+4.pdf` (11 pages), supplied for Issue #32.

Repository: yiipunn/TockTickIT-Service-Desk **Report state:** Staging evidence assembled on 2026-11-01 (Asia/Bangkok). The release to `main` and final submission remain pending.

This report follows the handout's required nine-part order. It links to the repository source of truth and uses existing screenshots and test reports. Where final-main evidence or personal student input is unavailable, the gap is stated explicitly.

Answer Part 1

Git Use with Engineering Workflow

The Lab 4 engineering contract was proposed in PR #64, followed by implementation and verification PRs #65, #66, #67, #68, #69, #70, #71, and #72. GitHub records show these nine PRs merged into `lab4-staging`. The peer-review record links each recorded approval by `DEV4952`, plus available author acknowledgements. It does not infer unrecorded review fixes.

The README provides setup, role, API, and test instructions. The `.gitignore` excludes dependencies, local environment files, build output, attachment storage, and transient test results. The repository contains the Lab 4 contract and records, server tests, client tests, E2E tests, and screenshots.

Still needed for final submission: a reviewed Issue #32 PR, the final `lab4-staging` → `main` PR and merge, final GitHub Project/Kanban evidence showing all issues Done, and final `main` commit history. These are not claimed here.

Answer Part 2

Spec DD

The rendered engineering contract contains numbered FR-01–FR-26, BR-01–BR-41, and AC-01–AC-29; a full Ticket transition matrix; Action lifecycle and resolution rules; dashboard calculations; data and migration decisions; API summary; and Product Definition of Done. The detailed UI specification and API specification refine those contracts. PR #64 is the pre-implementation specification PR; later implementation PRs begin with #65.

The data increment normalizes `ActionTaken` and append-only Action/Ticket events, keeps separate Ticket owner, Action creator, assignee, and performer identities, and uses integer versions for stale-write conflicts. The additive migration preserves earlier Users, Tickets, Attachments, Comments, and Notes. Legacy Tickets start with zero Actions; historic Resolved/Closed Tickets receive a documented `resolvedAt` proxy, without invented historic actors. Repeated seed runs are designed to converge on stable keys. Recovery after Lab 4 writes uses backup/forward repair rather than dropping history tables. See data decisions.

Answer Part 3

Test DD and Traceability



This XML file does not appear to have any style information associated with it. The document tree is shown below.


```
<testsuites name="vitest tests" tests="244" failures="0" errors="0" time="20.1078126">
  <testsuite name="tests/lab-01/categories.test.ts" timestamp="2026-10-30T05:42:33.234Z" tests="1" failures="0" errors="0" skipped="0" time="0.2468266">
    <testcase classname="tests/lab-01/categories.test.ts" name="GET /api/categories > returns the four seeded categories in id order" time="0.0175595"> </testcase>
  </testsuite>
  <testsuite name="tests/lab-01/health.test.ts" timestamp="2026-10-30T05:42:33.239Z" tests="1" failures="0" errors="0" skipped="0" time="0.0256086">
    <testcase classname="tests/lab-01/health.test.ts" name="GET /api/health > returns 200 with status ok and the service name" time="0.0241156"> </testcase>
  </testsuite>
  <testsuite name="tests/lab-02/attachments.test.ts" timestamp="2026-10-30T05:42:33.259Z" tests="24" failures="0" errors="0" skipped="0" time="0.4707021">
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > uploads an allowed attachment to an owned Ticket" time="0.0299746">
    </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > accepts PDF attachments" time="0.0118137"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > accepts evidence.jpg attachments" time="0.0088168"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > accepts evidence.webp attachments" time="0.0094317"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > downloads the persisted content of an owned active attachment"
    time="0.0064644"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > returns owned active attachment metadata" time="0.0063588"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects an unsupported attachment type" time="0.0051553"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects content that does not match its claimed allowed MIME type"
    time="0.0055364"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > accepts an attachment exactly 5 MB" time="0.0234875"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects an attachment larger than 5 MB" time="0.0190937"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects upload when no file is provided" time="0.0059106"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects a sixth active attachment" time="0.0103318"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > uses authenticated Requester context for upload" time="0.0114704">
    </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > does not allow a Requester to upload to another Requester's Ticket"
    time="0.0052466"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > soft-removes an owned active attachment" time="0.0117062"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > retains soft-removed attachment metadata in Ticket Detail"
    time="0.0352113"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > does not allow a Requester to remove another Requester's attachment"
    time="0.006172"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > does not expose another Requester's attachment through metadata"
    time="0.005176"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > does not expose another Requester's attachment through download"
    time="0.0060403"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > atomically enforces the active limit for concurrent uploads"
    time="0.0641649"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > returns 409 when removing an already removed attachment" time="0.0063719">
    </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects a missing removal reason" time="0.0100099"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects a whitespace-only removal reason" time="0.0063601"> </testcase>
    <testcase classname="tests/lab-02/attachments.test.ts" name="Lab 2 - Attachment Management API > rejects a overlong removal reason" time="0.0062063"> </testcase>
  </testsuite>
  <testsuite name="tests/lab-02/create-ticket.test.ts" timestamp="2026-10-30T05:42:33.269Z" tests="12" failures="0" errors="0" skipped="0" time="0.4536036">
    <testcase classname="tests/lab-02/create-ticket.test.ts" name="Lab 2 - Create Ticket API > creates a ticket for the authenticated Requester" time="0.0285107"> </testcase>
    <testcase classname="tests/lab-02/create-ticket.test.ts" name="Lab 2 - Create Ticket API > generates a different Ticket Number for each ticket" time="0.0330797">
    </testcase>
    <testcase classname="tests/lab-02/create-ticket.test.ts" name="Lab 2 - Create Ticket API > creates without a Development Requester header" time="0.0151392"> </testcase>
    <testcase classname="tests/lab-02/create-ticket.test.ts" name="Lab 2 - Create Ticket API > ignores a client-supplied inactive Development Requester" time="0.0162475">
    </testcase>
```


Server regression report: 244 tests, zero failures. Click for the source image.

The test plan and AC traceability matrix maps AC-01–AC-29 to unit, API/integration, component, style, authorization, workflow, migration, seed, performance-smoke, regression, and browser tests. Actual test paths are in its matrix and in the repository. Its initial `Planned / Not Run` entries record the planning state; execution records in §7 and this report describe later runs. Planned standalone unit files for Action validation/workflow and dashboard policy are absent; their behavior has varying API/component coverage. The responsive/accessibility browser suite was implemented at `client/e2e/lab-04/responsive-accessibility.spec.ts`, rather than the planned root `e2e/lab-04/` path.

The Issue #30 record reports 33 server files/244 tests, 18 client files/139 tests, successful production builds, six Lab 3 browser continuity tests, two responsive/accessibility browser tests, and two local performance-smoke tests. The Issue #31 record reports the same server/client totals and builds, six fresh migrations, a twice-run count-stable seed in an isolated schema, four Lab 4 browser tests, and four Lab 3 continuity tests. The checked-in Lab 4 JSON report and Lab 3 JSON report each record four expected, zero unexpected, zero skipped, and zero flaky results.

The genuine server test screenshot, client test screenshot, and performance smoke screenshot supplement the text record. On the Issue #32 branch, both production builds passed and the client suite passed 18 files/139 tests on 2026-11-01. Server tests were not rerun because the full suite uses the configured PostgreSQL database; this documentation pass did not modify a shared database. The handout's complete passing output from final `main` is still required after release.

Answer Part 4

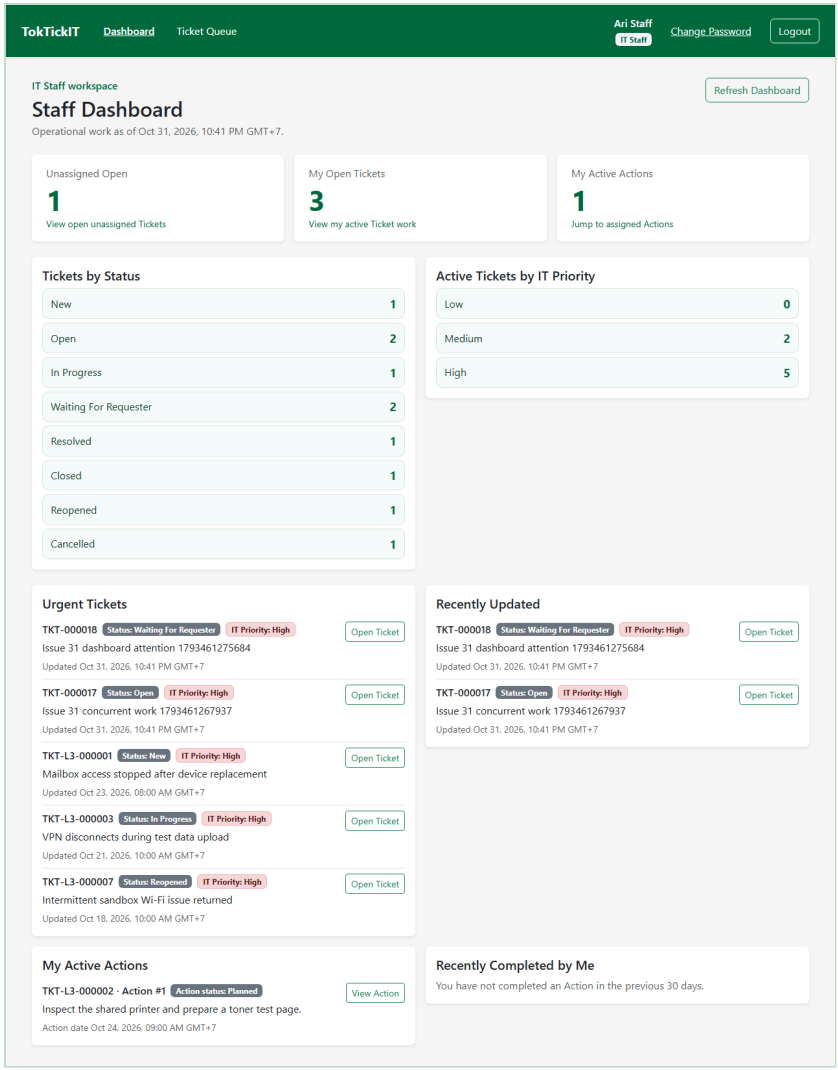
AI Use with Reflection

The AI-use record names OpenAI Codex/GPT-6 for the verified Issue #32 session, gives three actual user requests visible in this session, explains how AI-assisted documentation was checked, and records the limits of the available prompt history. It does not assign an unverified model or invented prompt to earlier PRs.

Submission gap: The handout asks for 6–10 selected prompts across Lab 4 and a brief first-person “My Reflection” about specification-agent and coding-agent use. Earlier prompt transcripts and the student's own reflection are not available in the repository or GitHub reviews. The student must add them to `ai-use.md` and regenerate this PDF before submission.

Answer Part 5

Working IT Staff Dashboard UI



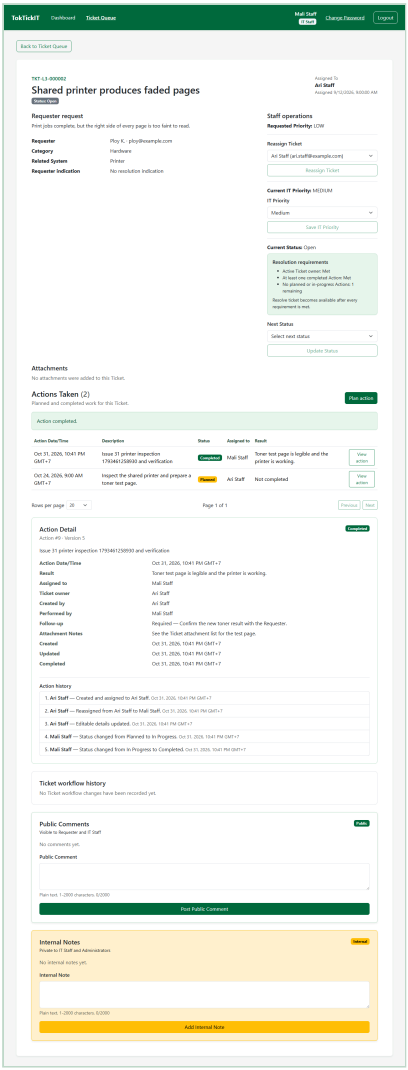
IT Staff Dashboard from the Issue #31 browser run. Click for the source image.

The Staff Dashboard screenshot shows unassigned open Tickets, my open Tickets, my active Actions, counts by status and IT Priority, urgent and recently updated Tickets, assigned Actions, and the empty recently-completed list. The contract specifies active status membership, current-user scopes, UTC windows, tie ordering, bounded lists, and zero-key behavior. Cards and rows lead to Ticket Queue, Ticket Detail, or Action Detail.

The Staff Dashboard API tests use fixture data to compare authoritative counts, scopes, order, empty states, Admin reuse, and safe failures. The Issue #31 E2E scenario compares selected rendered metrics with the dashboard API response and follows representative drill-downs. The Issue #28 API-test screenshot is an additional genuine test artifact. No independent SQL-query screenshot is present, so the final handout's selected metric-to-database-query demonstration remains to be captured.

Answer Part 6

Working Actions Taken UI



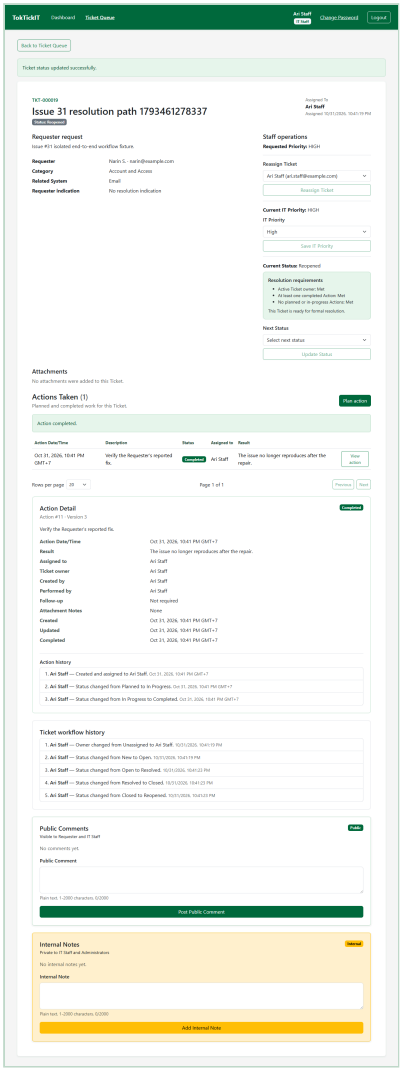
Actions Taken and Action history on one Ticket. Click for the source image.

The Actions Taken flow screenshot shows two Actions on one Ticket, including a completed Action with creator, assignee, performer, Ticket owner, follow-up, attachment notes, and ordered Action history. The cancellation screenshot supplies a separate terminal-state example. The Action model requires date/time and description, validates a conditional follow-up note, and requires a Result on completion. Assignment accepts active Staff/Admin users; Requesters see current Actions on owned Tickets read-only.

The Action API tests, component tests, and browser flow cover creation, editing, reassignment, transitions, validation, inactive-assignee rejection, role/ownership protection, stale-write conflicts, retained draft, and safe errors. The UI specification describes loading, empty, validation, conflict, and terminal read-only behavior. The browser evidence is representative, not an exhaustive visual capture of each negative state.

Answer Part 7

Working Ticket Workflow



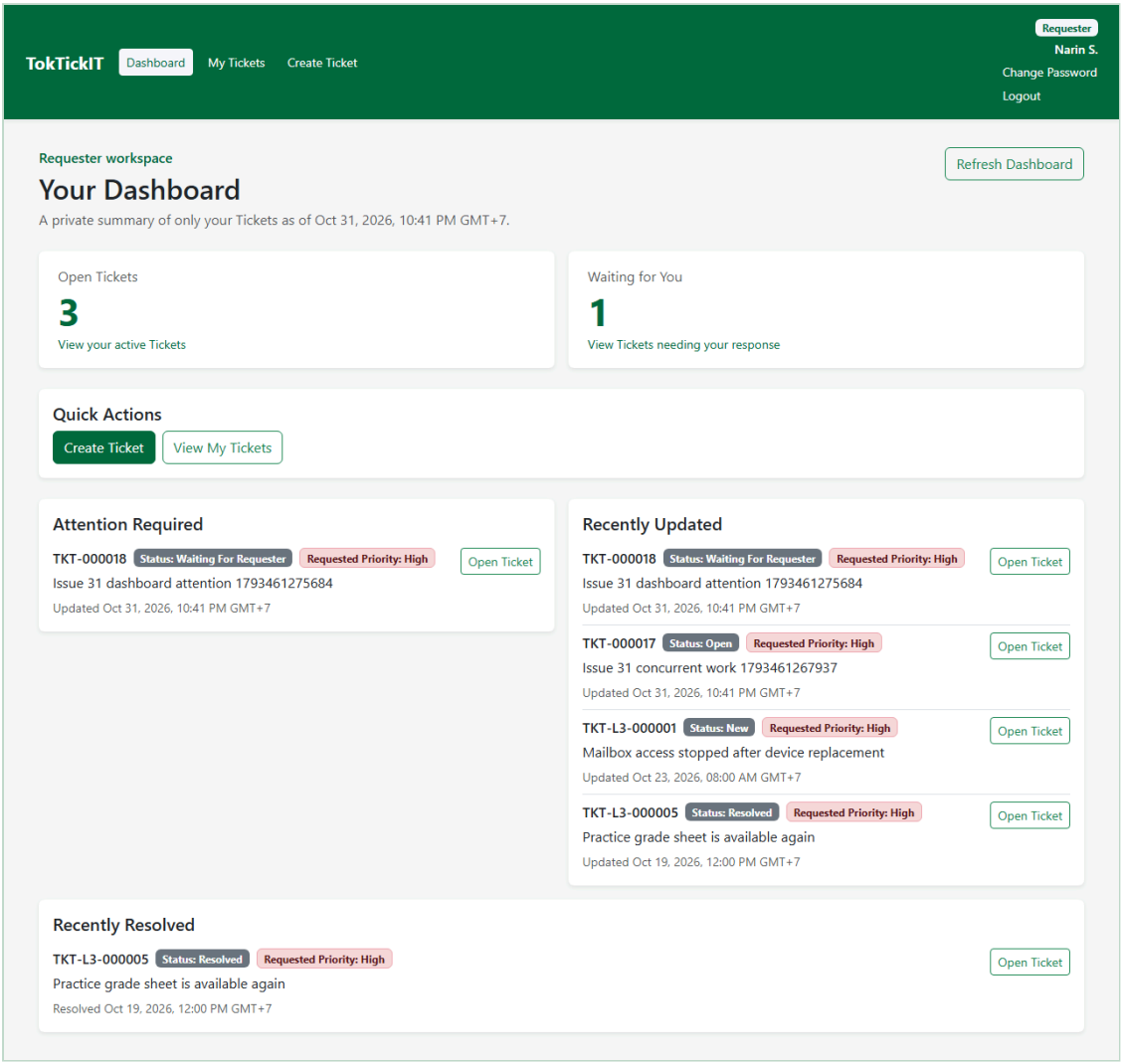
Formal Ticket resolution workflow. Click for the source image.

The transition matrix defines permitted moves among New, Open, In Progress, Waiting for Requester, Resolved, Closed, Reopened, and Cancelled. Backend resolution requires an active Ticket owner, at least one completed Action, and no planned or in-progress Actions. A Requester's “problem appears resolved” indication is advisory. Ticket owner/status mutations append workflow events and use `expectedVersion`; stale writes return a conflict without overwriting the winner.

The Ticket resolution screenshot, workflow API tests, component tests, and browser scenario show the gate, formal resolution, close/reopen path, and role-appropriate history. Cancellation of remaining work is covered by Action API/component tests rather than that browser scenario.

Answer Part 8

Working Requester Dashboard and Final Regression UI



Requester Dashboard from the Issue #31 browser run. Click for the source image.

The Requester Dashboard screenshot shows owned open/waiting counts, attention-required Tickets, recently updated Tickets, and recently resolved Tickets. The dashboard contract limits all Requester queries to the authenticated owner, uses inclusive 30-day UTC windows, and defines empty keys and deterministic ordering. The Requester API tests check counts and cross-owner denial; the component tests check states and drill-down. The Issue #31 browser scenario checks selected metrics and links.

Lab 1–3 regression evidence is the Issue #30 full server/client runs and the Issue #31 four-scenario Lab 3 browser continuation. The suite includes health/reference data, authentication/session and password change, Requester creation/My Tickets/Detail, attachments, Public Comments, Staff queue/claim/assignment/priority/status, Internal Notes, and Administrator User Management. See the test execution record, server test screenshot, and client test screenshot. This is representative automated regression, not proof that every earlier manual screen was revisited.

Answer Part 9

Zen Green UI, Responsive, Accessibility, and Final Polish

The rendered UI specification defines the shared Zen Green shell, role navigation, status/private-content cues, loading and failure feedback, responsive layouts, and accessibility checklist. Genuine desktop, tablet, mobile, 320 px reflow, 200% zoom, and keyboard focus artifacts come from the Issue #30 browser inspection. The Issue #31 mobile screenshot adds integrated Lab 4 evidence.

The visual/accessibility checklist remains in `ui-spec.md` §10–11. Evidence supports automated/component checks, keyboard focus, viewport reflow, and screenshot review; a complete screen-reader audit and every manual checklist item are not documented. Avoid marking those unchecked items complete. Review the final PDF at normal zoom and refresh screenshots only from a running app if any are replaced.

Outstanding submission items

1. Finish Issue #32 peer review, then release `lab4-staging` to `main` through a separate reviewed PR.
2. Run the complete suites on final `main`; capture readable output and record commit, environment, and results.
3. Add the student's 6–10 actual AI prompts and first-person reflection; add verified reviewer name/student ID if required.
4. Record final Kanban Done state, main-branch commit chain, and selected dashboard metric-to-database query comparison.
5. Regenerate and submit exactly one final PDF after those items are complete.